Git et GitHub

Sauvegarder, suivre et présenter son travail comme un développeur. Les trois zones de Git, le commit, l'historique, les branches, le dépôt distant GitHub, et comment revenir en arrière sans rien perdre.

1. Pourquoi versionner
2. Les trois zones de Git
3. Le cycle de base
4. Écrire un bon commit
5. Le fichier .gitignore
6. Les branches
7. GitHub et le dépôt distant
8. Revenir en arrière
9. Les pièges réels
10. Exercices corrigés
11. Mission : ExamPredict sur GitHub
12. À savoir par cœur

---

1. Pourquoi versionner

Définition 1.1 *(gestion de versions)*

Un **système de gestion de versions** enregistre l'historique complet des modifications d'un projet : qui a changé quoi, quand et pourquoi. **Git** est ce système. Il fonctionne **sur ton ordinateur**.

Définition 1.2 *(Git ou GitHub)*

Git est un programme local. **GitHub** est un site web qui héberge des dépôts Git en ligne, pour les sauvegarder, les partager et les montrer.

En clair : Git est l'appareil photo, GitHub est l'album en ligne.

Dans le métier

Une équipe ML modifie le même code tous les jours. Sans Git, impossible de savoir quelle version du code a produit tel modèle, ni de revenir à la version qui marchait hier. Et ton GitHub est ton CV de développeur : un recruteur y regarde tes projets, tes tests et la régularité de tes commits.

2. Les trois zones de Git

Définition 2.1

Un fichier suivi par Git peut se trouver dans trois zones :

ZONE	RÔLE	ANALOGIE
Répertoire de travail	les fichiers tels que tu les modifies	la pièce où tu ranges
Index (*staging area*)	ce qui sera dans la prochaine photo	les gens que tu places devant l'objectif
Dépôt (*repository*, dossier `.git`)	l'historique des photos (commits)	l'album photo

```
répertoire de travail ──git add──►  index  ──git commit──►  dépôt local ──git p
```

Proposition 2.2

Un commit enregistre **uniquement** ce qui se trouve dans l'index. Un fichier modifié mais non ajouté avec `git add` n'est pas dans la photo.

3. Le cycle de base

COMMANDE	EFFET
`git init`	crée un dépôt dans le dossier courant (une seule fois par projet)
`git status`	montre l'état des fichiers : non suivis, modifiés, dans l'index
`git add fichier.py`	place un fichier dans l'index (`git add .` pour tout le dossier)
`git commit -m "message"`	prend la photo de l'index, avec une légende
`git log --oneline`	affiche l'historique, un commit par ligne
`git diff`	montre les lignes modifiées et pas encore ajoutées

Méthode 3.1 *(la routine du développeur)*

```
git status                          # 1. qu'est-ce qui a changé ?
git add src/analyzer.py             # 2. je choisis ce que je photographie
git status                          # 3. je vérifie (fichiers en vert)
git commit -m "Corrige la détection du mot-clé ph"
git push                            # 4. j'envoie sur GitHub
```

Remarque 3.2 *(lire git status)*

En **rouge** : modifié ou nouveau, pas encore dans l'index. En **vert** : dans l'index, prêt pour le commit. « nothing to commit, working tree clean » : tout est sauvegardé.

4. Écrire un bon commit

Méthode 4.1 *(un commit = une idée)*

Un commit regroupe des modifications qui forment **une seule idée**. Le message commence par un **verbe** et dit **ce que fait** le changement.

❌ MAUVAIS	✅ BON
`update`	`Ajoute les tests de count_keyword`
`fix`	`Corrige le plantage quand le dossier data est vide`
`trucs divers`	`Lit les PDF avec gestion des fichiers corrompus`

En clair : dans six mois, toi ou un collègue devez comprendre l'historique sans ouvrir le code.

5. Le fichier `.gitignore`

Définition 5.1

`.gitignore` liste les fichiers et dossiers que Git doit **ignorer** : ils n'apparaîtront jamais dans `git status` et ne seront jamais commités.

```
# .gitignore d'un projet Python / ML
__pycache__/          # fichiers générés automatiquement par Python
*.pyc
.venv/                # l'environnement virtuel (lourd, propre à ta machine)
.pytest_cache/
data/                 # les datasets (lourds, parfois confidentiels)
.env                  # les clés secrètes (API, mots de passe)
```

Piège 5.2 *(les secrets)*

Une clé d'API poussée sur un dépôt public est trouvée par des robots en quelques minutes. Les secrets vont dans un fichier `.env`, listé dans le `.gitignore`, **avant** le premier commit. Effacer la clé dans un commit suivant ne suffit pas : elle reste dans l'historique.

6. Les branches

> **Définition 6.1**
>
> Une **branche** est une ligne de développement parallèle. La branche principale s'appelle `main`. On crée une branche pour essayer une idée sans risquer de casser `main`.
>
> **En clair :** un brouillon. Si l'idée est bonne, on la recopie au propre (on **fusionne**). Sinon, on jette le brouillon.

```
git switch -c essai-tfidf      # crée la branche et s'y place
# ... modifications + commits ...
git switch main                # retour sur la branche principale
git merge essai-tfidf          # fusionne le travail du brouillon dans main
git branch                     # liste les branches (* = branche actuelle)
```

7. GitHub et le dépôt distant

> **Définition 7.1** *(remote)*
>
> Un **dépôt distant** (*remote*) est une copie du dépôt hébergée ailleurs, par exemple sur GitHub. Par convention, il s'appelle `origin`.

COMMANDE	EFFET
`git remote add origin URL`	relie le dépôt local à GitHub (une seule fois)
`git push -u origin main`	premier envoi ; ensuite, `git push` suffit
`git pull`	récupère les commits faits ailleurs
`git clone URL`	télécharge un dépôt complet sur une nouvelle machine
`git remote -v`	affiche les dépôts distants configurés

Remarque 7.2 *(HTTPS ou SSH)*

Une adresse `https://github.com/…` s'authentifie par ton compte GitHub. Une adresse `git@github.com:…` (SSH) demande une **clé SSH** enregistrée sur GitHub. C'est pour ça que ton premier push en SSH a été refusé (`Permission denied (publickey)`) et qu'il est passé en HTTPS.

8. Revenir en arrière

SITUATION	COMMANDE
J'ai abîmé un fichier, pas encore ajouté : je veux la dernière version commitée	`git restore fichier.py`
J'ai ajouté un fichier à l'index par erreur	`git restore --staged fichier.py`
Un commit déjà poussé est mauvais : je l'annule proprement	`git revert <id_du_commit>` (crée un commit inverse)
Voir le contenu d'un ancien commit	`git show <id_du_commit>`

Piège 8.1

`git restore fichier.py` efface tes modifications non commitées **définitivement**. Fais `git diff` avant, pour voir ce que tu vas perdre.

9. Les pièges réels

Le dépôt dans un dépôt (ton cas)

`AI-ENGINEER-2027` est un dépôt, et `exampredict-ai` à l'intérieur en est un autre. Le dépôt parent voit le dossier enfant comme un « dépôt imbriqué » et ne suit pas son contenu correctement. Solution : chaque projet a son propre dépôt GitHub, et le parent ignore l'enfant (`exampredict-ai/` dans son `.gitignore`). C'est ce qui a été fait.

« nothing added to commit »

Tu as oublié `git add` (Proposition 2.2). Et si « Author identity unknown » s'affiche, configure une fois pour toutes `git config --global user.name "…"` et `git config --global user.email "…"`.

10. Exercices corrigés

Exercice 1 · L'ordre des zones ★

Remettre dans l'ordre : `git commit`, `git push`, modifier le fichier, `git add`.

Corrigé. Modifier → `git add` → `git commit` → `git push`. Répertoire de travail → index → dépôt local → GitHub.

Exercice 2 · Diagnostic ★★

Tu modifies `main.py`, tu fais `git commit -m "Ajoute l'affichage"`, et Git répond « no changes added to commit ». Pourquoi ?

Corrigé. `main.py` n'est pas dans l'index : il manque `git add main.py` avant le commit.

Exercice 3 · Le bon .gitignore ★★

Parmi `src/`, `.venv/`, `README.md`, `__pycache__/`, `.env` et `tests/`, que faut-il ignorer ?

Corrigé. `.venv/`, `__pycache__/` et `.env`. Le code, les tests et le README doivent être sur GitHub.

Exercice 4 · Réécrire des messages ★★

Améliorer : « modif », « ça marche enfin », « ajout ».

Corrigé (exemples). « Remplace text.count par une recherche par mot entier », « Corrige la lecture des fichiers vides », « Ajoute la fonction de calcul des statistiques ».

11. Mission : ExamPredict sur GitHub

> **À faire au terminal (Git Bash)**
>
> 1. `cd ~/AI-ENGINEER-2027/exampredict-ai` puis `git status` : lis chaque fichier rouge.
> 2. Remplis le `.gitignore` (section 5) et vérifie avec `git status` que `.venv` et `__pycache__` ont disparu.
> 3. `git branch -M main`, puis `git add .`, puis `git status` : aucun fichier PDF ni `.venv` en vert.
> 4. Premier commit avec un message à la Méthode 4.1.
> 5. Crée le dépôt `exampredict-ai` sur GitHub (public, vide), puis `git remote add origin https://github.com/joe-web-startup/exampredict-ai.git` et `git push -u origin main`.
> 6. Crée une branche `fix-ph`, branche `count_keyword` à la ligne 149, commite, reviens sur `main`, fusionne, puis pousse.

12. À savoir par cœur

1	Git est l'appareil photo (local), GitHub est l'album (en ligne).
2	Les trois zones : répertoire de travail → `add` → index → `commit` → dépôt → `push` → GitHub.
3	Un commit ne contient que ce qui est dans l'index.
4	`git status` avant et après chaque action : rouge = pas ajouté, vert = prêt.
5	Un commit = une idée, avec un message qui commence par un verbe.
6	`.gitignore` : `.venv`, `__pycache__`, les données lourdes et les secrets. Jamais de clé d'API sur GitHub.
7	Branche = brouillon. `git switch -c`, puis `git merge`.
8	`remote add origin` une fois, `push -u origin main` la première fois, puis `push`.
9	`git restore` annule des modifications locales, `git revert` annule un commit publié.